



UNIVERSIDAD DE GRANADA

Periféricos y Dispositivos de Interacción Humana

Práctica 2: NCurses

Pablo Soriano Fleitas

Programas de ejemplo

Ejemplo 1: Hello

Código:

```
#include <ncurses.h>
int main() {
    initscr();           // Inicializar modo curses
    printw("Hello, World!"); // Imprimir mensaje
    refresh();           // Mostrar mensaje por pantalla
    getch();             // Esperar la pulsación de una tecla
    endwin();            // Terminar modo curses

    return 0;
}
```

Compilación:

```
pablo@PortatilPablito:/mnt/c/NcursesPDIH$ gcc hello.c -o hello -lncurses
```

Ejecución:



A screenshot of a terminal window. The title bar shows 'pablo@PortatilPablito: /mnt/c' with standard window controls. The terminal content displays 'Hello, World!' followed by a cursor.

Ejemplo 2: Ventana

Código:

```
#include <stdlib.h>
#include <ncurses.h>
int main(void) {
    int rows, cols;

    initscr();

    if (has_colors() == FALSE) {
        endwin();
        printf("El terminal no tiene soporte de color \n");
        exit(1);
    }

    start_color();
    init_pair(1, COLOR_YELLOW, COLOR_GREEN);
    init_pair(2, COLOR_BLACK, COLOR_WHITE);
    init_pair(3, COLOR_WHITE, COLOR_BLUE); // blanco sobre fondo azul
    clear();

    refresh();
    getmaxyx(stdscr, rows, cols);

    WINDOW *window = newwin(rows, cols, 0, 0);
    wbgd(window, COLOR_PAIR(3));
    box(window, '|', '-');

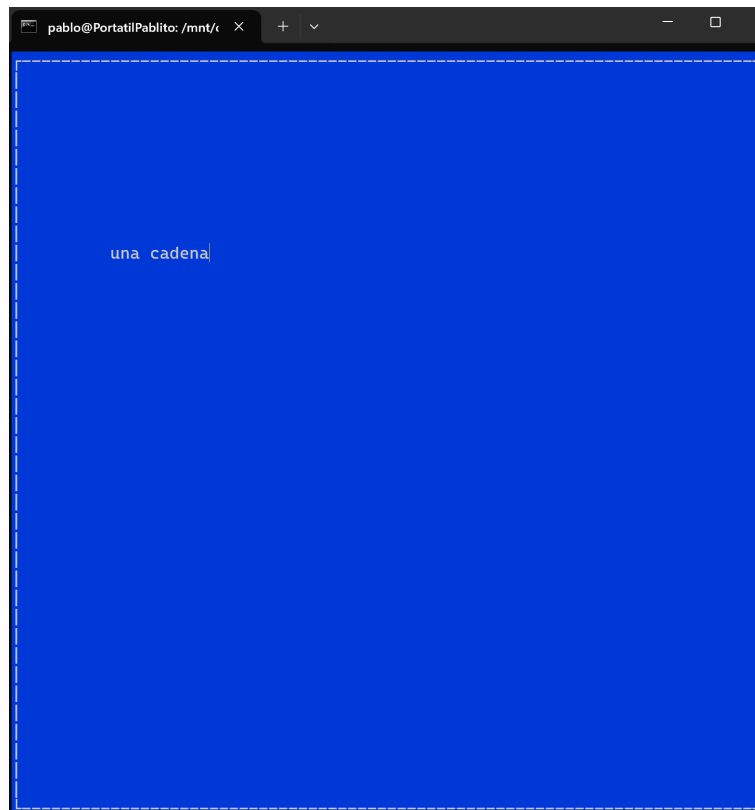
    mvwprintw(window, 10, 10, "una cadena");
    wrefresh(window);

    getch();
    endwin();
    return 0;
}
```

Compilación:

```
pablo@PortatilPablito:/mnt/c/NcursesPDIH$ gcc ventana.c -o ventana -lncurses
```

Ejecución:



Ejemplo 3: Pelotita

Código:

```
#include <ncurses.h>
#include <unistd.h>

#define DELAY 30000

int main(int argc, char *argv[]) {
    int x = 0, y = 0;
    int max_y = 50, max_x = 50;
    int next_x = 0;
    int direction = 1;

    initscr();
    noecho();
    curs_set(FALSE);

    while(1) {
        clear();
        mvprintw(y, x, "o");
        refresh();

        usleep(DELAY);

        next_x = x + direction;

        if (next_x >= max_x || next_x < 0) {
            direction *= -1;
        } else {
            x += direction;
        }
    }

    endwin();
}
```

Compilación:

```
pablo@PortatilPablito:/mnt/c/NcursesPDIH$ gcc pelotita.c -o pelotita -lncurses
```

Ejecución:



Juego Sencillo: Snake

En la guía se nos propone un juego estilo “pong”, pero si no estoy equivocado, se supone que podíamos crear algo distinto que no tiene que ser exactamente ese juego basándonos en el programa de ejemplo de movimiento de pelotas, había pensado en el juego del snake, donde la “pelota” moviéndose sería la propia serpiente que se va haciendo más grande a medida que va comiéndose las “manzanas” que he pensado en hacerlas como pelotas que van saliendo en zonas random de color rojo.

Código:

```
#include <ncurses.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define FRAME_DELAY_US 90000

typedef struct {
    int y;
    int x;
} Point;
```

```
} else {
    y = rows / 2 - 4;
    if (y < 1) {
        y = 1;
    }

    x = (cols - 18) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y, x, "Snake con ncurses");

    x = (cols - 28) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y + 2, x, "Autor: Pablo Soriano Flietas");

    x = (cols - 10) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y + 4, x, "Controles:");

    x = (cols - 28) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y + 5, x, "Flechas -> mover serpiente");

    x = (cols - 30) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y + 6, x, "Q -> terminar partida actual");

    x = (cols - 32) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y + 7, x, "Comida (@ roja) -> suma longitud");

    x = (cols - 29) / 2;
    if (x < 1) {
        x = 1;
    }
    mvprintw(y + 9, x, "S/Enter: empezar Q: salir");
}
```

```
static bool show_welcome_screen(void) {
    int rows, cols;
    int y;
    int x;
    int key;

    nodelay(stdscr, FALSE);

    clear();
    getmaxyx(stdscr, rows, cols);
    box(stdscr, 0, 0);

    if (rows < 12 || cols < 46) {
        y = rows / 2 - 1;
        if (y < 1) {
            y = 1;
        }
        x = (cols - 30) / 2;
        if (x < 1) {
            x = 1;
        }
        mvprintw(y, x, "Snake con ncurses");
        if (y + 1 < rows - 1) {
            mvprintw(y + 1, x, "S/Enter: jugar Q: salir");
        }
    }
}
```

```
refresh();
while (1) {
    key = getch();
    if (key == 's' || key == 'S' || key == '\n' || key == KEY_ENTER) {
        nodelay(stdscr, TRUE);
        return true;
    }
    if (key == 'q' || key == 'Q') {
        return false;
    }
}

return false;
```

Esta primera parte del código es una función booleana con la que creo la página de bienvenida para el jugador que abre el juego con el texto que se muestra por pantalla y la opción “S/s” para empezar a jugar o “Q/q” para salir.

```
static bool is_on_snake(const Point *snake, int length, int y, int x) {
    for (int i = 0; i < length; i++) {
        if (snake[i].y == y && snake[i].x == x) {
            return true;
        }
    }
    return false;
}

static Point random_food(const Point *snake, int length, int rows, int cols) {
    Point food;

    do {
        food.y = (rand() % (rows - 2)) + 1;
        food.x = (rand() % (cols - 2)) + 1;
    } while (is_on_snake(snake, length, food.y, food.x));

    return food;
}
```

La función is on snake la uso para comprobar que el píxel de la pantalla no coincide con uno de la serpiente a la hora de usar random food para colocar de forma aleatoria un símbolo rojo como comida para el juego.

```
static int run_game(void) {
    int direction_y = 0;
    int direction_x = 1;
    int key;
    bool game_over = false;
    bool use_color;
    WINDOW *gamewin;

    getmaxyx(stdscr, rows, cols);
    use_color = has_colors();

    if (rows < 10 || cols < 20) {
        clear();
        mvprintw(rows / 2, (cols - 45) / 2, "Ventana demasiado pequena. Minimo recomendado: 20x10");
        mvprintw(rows / 2 + 1, (cols - 29) / 2, "Pulsa una tecla para continuar");
        nodelay(stdscr, FALSE);
        refresh();
        getch();
        nodelay(stdscr, TRUE);
        return 0;
    }

    gamewin = newwin(rows, cols, 0, 0);
    if (gamewin == NULL) {
        nodelay(stdscr, FALSE);
        clear();
        mvprintw(rows / 2, (cols - 38) / 2, "No se pudo crear la ventana principal");
        mvprintw(rows / 2 + 1, (cols - 29) / 2, "Pulsa una tecla para continuar");
        refresh();
        getch();
        nodelay(stdscr, TRUE);
        return 0;
    }

    if (use_color) {
        wbkgd(gamewin, COLOR_PAIR(2));
    } else {
        wbkgd(gamewin, A_NORMAL);
    }

    keypad(gamewin, TRUE);
    nodelay(gamewin, TRUE);

    int max_cells = (rows - 2) * (cols - 2);
    Point *snake = malloc((size_t)max_cells * sizeof(Point));
    if (snake == NULL) {
        delwin(gamewin);
        nodelay(stdscr, FALSE);
        clear();
        mvprintw(rows / 2, (cols - 43) / 2, "No se pudo reservar memoria para la serpiente");
        mvprintw(rows / 2 + 1, (cols - 29) / 2, "Pulsa una tecla para continuar");
        refresh();
        getch();
        nodelay(stdscr, TRUE);
        return 0;
    }

    int length = 1;
    snake[0].y = rows / 2;
    snake[0].x = cols / 2;

    Point food = random_food(snake, length, rows, cols);

    while (!game_over) {
        key = wgetch(gamewin);

        if (key == 'q' || key == 'Q') {
            game_over = true;
            continue;
        }

        if (key == KEY_UP && direction_y != 1) {
            direction_y = -1;
            direction_x = 0;
        } else if (key == KEY_DOWN && direction_y != -1) {
            direction_y = 1;
            direction_x = 0;
        } else if (key == KEY_LEFT && direction_x != 1) {
            direction_y = 0;
            direction_x = -1;
        } else if (key == KEY_RIGHT && direction_x != -1) {
            direction_y = 0;
            direction_x = 1;
        }

        int new_head_y = snake[0].y + direction_y;
        int new_head_x = snake[0].x + direction_x;

        if (new_head_y <= 0 || new_head_y >= rows - 1 || new_head_x <= 0 || new_head_x >= cols - 1) {
            game_over = true;
            continue;
        }
    }
}
```

```
bool ate_food = (new_head_y == food.y && new_head_x == food.x);

if (ate_food && length < max_cells) {
    for (int i = length; i > 0; i--) {
        snake[i] = snake[i - 1];
    }
    length++;
} else {
    for (int i = length - 1; i > 0; i--) {
        snake[i] = snake[i - 1];
    }
}

snake[0].y = new_head_y;
snake[0].x = new_head_x;

if (ate_food) {
    food = random_food(snake, length, rows, cols);
}

werase(gamewin);
box(gamewin, '|', '-');
mvwprintw(gamewin, 0, 2, " Longitud: %d ", length);
if (cols > 20) {
    mvwprintw(gamewin, 0, cols - 16, " Q para salir ");
}

if (use_color) {
    wattron(gamewin, COLOR_PAIR(1));
    mvwaddch(gamewin, food.y, food.x, '@');
    wattroff(gamewin, COLOR_PAIR(1));
} else {
    mvwaddch(gamewin, food.y, food.x, '@');
}

mvwaddch(gamewin, snake[0].y, snake[0].x, 'O');
for (int i = 1; i < length; i++) {
    mvwaddch(gamewin, snake[i].y, snake[i].x, 'o');
}

wrefresh(gamewin);
usleep(FRAME_DELAY_US);
}

free(snake);
delwin(gamewin);
touchwin(stdscr);
refresh();
return length - 1;
}
```

Esta otra función es la principal del juego, donde se pone en marcha el juego en una pantalla aparte que es la que llamo gamewin y está corriendo hasta que se acabe la partida, que es o cuando pulso la "Q/q" o cuando se choca contra uno de los bordes de la pantalla.

```
static bool show_summary_screen(int score) {
    int rows, cols;
    int key;

    nodelay(stdscr, FALSE);
    clear();
    getmaxyx(stdscr, rows, cols);

    box(stdscr, 0, 0);
    mvprintw(rows / 2 - 2, (cols - 20) / 2, "Fin de la partida");
    mvprintw(rows / 2 - 1, (cols - 26) / 2, "Marcador final: %d", score);
    mvprintw(rows / 2, (cols - 32) / 2, "Felicidades, gran partida!");
    mvprintw(rows / 2 + 2, (cols - 44) / 2, "Pulsa R para jugar de nuevo o Q para salir");
    refresh();

    while (1) {
        key = getch();
        if (key == 'r' || key == 'R') {
            nodelay(stdscr, TRUE);
            return true;
        }
        if (key == 'q' || key == 'Q') {
            return false;
        }
    }
}
```

Esta última es la función que llama a la pantalla final cuando se acaba una partida donde se puede o bien volver a jugar, o salir y se puede ver la puntuación obtenida.

```

int main(void) {
    bool play_again;
    int score;

    initscr();
    noecho();
    cbreak();
    curs_set(FALSE);
    nodelay(stdscr, TRUE);
    keypad(stdscr, TRUE);
    srand((unsigned int)time(NULL));

    if (has_colors()) {
        start_color();
        init_pair(1, COLOR_RED, COLOR_BLACK);
        init_pair(2, COLOR_WHITE, COLOR_BLUE);
    }

    play_again = show_welcome_screen();

    while (play_again) {
        score = run_game();
        play_again = show_summary_screen(score);
    }

    endwin();
    return 0;
}

```

Este finalmente es el main, donde se inicia el juego y se llama a las funciones, usando el play again para que se siga jugando hasta que se decida que se para de jugar.

Compilación:

```
gcc snake.c -o snake -lncurses
```

Ejecución:

```

pablo@PortatilPablito: /mnt/c
Snake con ncurses
Autor: Pablo Soriano Flietas
Controles:
Flechas -> mover serpiente
Q -> terminar partida actual
Comida (@ roja) -> suma longitud
S/Enter: empezar Q: salir

```

